IB Computer Science

MRS GARMESON

g.garmeson@stgeorge.es

New syllabus - first exams May 2027

No more pseudocode. Written exams use Python

Theme A: Concepts of computer science
Paper 1: Topics A1, A2, A3, A4 + Case study (2 hours) - 40%

Theme B: Computational thinking and problem solving
Paper 2: Topics B1, B2, B3, B4 (2 hours) - 40%

Internal assessment: A programming project of the students own choice (35 hours of class time) 20% of grade

Theme A

A1 Computer Fundamentals

CPU structure, GPUs, pipelining, memory, fetch-decode-execute cycle), SaaS/PaaS, number systems, logic gates, truth tables, circuit diagrams, operating systems & resource management, control systems, compilers/interpreters

A2 Networks

Devices, client/server v peer, topologies, protocols (tcp/ip, ip4/ip6, http, dns), switching, routing, security vulnerabilities & countermeasures, encryption & certificates.

Theme A

A3 Databases

Schema, relational databases, ERD, 1NF/2NF/3NF, SQL including joins, update/insert, aggregate functions, views, transactions, data warehousing.

A4 Machine Learning

Hardware, types of ML, preprocessing, linear regression, classification, hyperparameters, unsupervised clustering, association rule, reinforcement learning, genetic algorithms, ANNs, CNNs,ethical challenges.

Theme B

B1 Computational thinking

Abstraction, algorithm design, decomposition, pattern recognition, trace flowcharts.

B2 Programming

Data types and their operations, selection, iteration, functions, static/dynamic structures, arrays/lists, stacks, queues, big O, sorting algorithms, recursion, file processing.

Theme B

B3 Object oriented programming

Classes/objects, constructors, static/non-static, encapsulation, inheritance, polymorphism, abstract classes, composition/aggregation, design patterns.

B4 Abstract data structures

Linked lists (singular, double, circular), binary search trees, hash-maps, sets.

Case study

An annually changing case study that prompts students to research into a field of emerging technology.

Planned Sequence

Year 12:

B2 Programming,
B1 Computational thinking

== Half term ==

B3 Object oriented programming

== Christmas ==

A3 Databases, B4 Abstract Data Types

== Easter ==

IA lessons

Year 13:

A1 Computer Fundamentals
A4 Machine Learning

== Half term ==

A4 Machine Learning
A2 Networks

== Christmas ==

Lesson 1

Set up github

Github - why?

- Collaboration & Version Control
- Keep a history of your code (like 'undo' for projects)
- Work together without overwriting each other’s files
- Build a portfolio you can show universities/employers

Github - basics

- Repository (repo): your project folder on GitHub
- Commit: a save point with a short message
- Push: upload your commits to GitHub.com
- Pull: download teammates’ latest changes

Github - workflow

- Clone → copy repo from GitHub to your computer
- Edit → make code changes in your IDE (e.g., VS Code)
- Commit → save your changes with a message
- Push → upload to GitHub
- Pull → get updates from teammates

Github - install

- Go to github.com and create an account
- Install an IDE such as VS Code

Github - first repository

1. On GitHub.com, click New Repository
2. Name it something simple (e.g., hello-world)
3. Add a README file
4. Keep it private (or public if you want)
5. Open GitHub Desktop → Clone a repository → select the repo you just made.
6. Open the repo in VS Code (button in GitHub Desktop) or CMD-shift-A
7. Create a simple file, e.g., hello.py or hello.java, that prints "Hello, GitHub!"
8. Back in GitHub Desktop, commit to main (write a short commit message, like "add hello program").
9. Click Push origin.
10. Refresh GitHub.com → you'll see their code online.

Github - collaborating

1. One student creates the repo → adds the other two as collaborators (Settings → Manage Access).
2. All three clone the repo in GitHub Desktop.
3. Before coding: Pull (get latest changes).
4. Make changes in their file or section.
5. Commit changes with a clear message (“add room descriptions”).
6. Push changes to GitHub.
7. Others Pull before continuing.

I will add you as collaborators on maths-game.

IGCSE recap #1 Variables / data types

Variable - a designated memory location that stores a value that can change during the execution of the program.

Data types: **String**

Integer

Decimal (float / real / double)

Boolean - True or False (only two values)

Char (not in Python) - a string of length 1

It is convenient in Python to not have to declare the data type of a variable but there is a performance cost to this

Identifier - name of a variable of other entities

It’s crucial that identifiers are meaningful and descriptive. Why?

Assignment - setting a value to a variable

The value on the right is assigned to the variable on the left e.g. count = 1

Increment to increase a value by another value (usually 1)

e.g. count = count + 1 is the same as count += 1

Decrement: same, but decrease

e.g. count -= 1 or count -= 2

Initialization - assigning an initial value to a data structure or variable

Must we initialize variables in python?
Yes, but we don't need to declare them.

Operator - a character that represents arithmetic, relational or logical operation

Arithmetic: +, -, *, /, % (modulus), // (floor division)

Relational: > , >=, <, <=, ==, !=

Boolean / logical: and, or, not

Tasks

#1 Ask the user for 3 numbers. Output the average.

#2 Ask the user for their name and age. Output their age in 10 years.

E.g. input “Bob”, 27 output “Bob, in 10 years you will be 37”

#3 Ask the user for a 3 digit number. Output the sum of its digits.

String manipulation

Concatenate (put strings together), replace, or alter strings

```
print(""" Write multiple

Lines like this

""")

exam = "CS"

grade = 79

print("Your", exam, "score is", grade)

f-strings: print(f"Your {exam} score is {grade}")
```

```
myString = “Hello year 12”

len(myString)

myString[7]

myString.find(“l”)

myString.rfind(“l”)
```

```
myString = “Hello year 11”

myString.replace(“11”,”12”)

myString.split()

myString.strip()

#substring/slicing

myString[0,4]       myString[:8]        myString[6:]

myString[-1]        myString[-6]        myString[-6:]

myString[6:-3]
```

TASK

Ask the user to enter their school username in the following format:

- Year Group Using two digits (e.g. “07” for year 7, “11” for year 11, “00” for staff members)
- 1 character for their initial (first letter of their firstname)
- The user’s lastname
- A 2-digit code: “_S” for students, “_T” for teachers, “_A” for admin staff.

07jFox_S
09kJohnson_S
00pJones_T
00jOliver_A

TASK

Validate the username. Your program should read the username and decide:

If the username is less than 6 characters long the program should ask the user to enter a valid username.
If the username does not contain the character "_" it should also ask the user to enter a valid username.
If the username is valid, the program should decide if the user is a member of staff or a student.
If they are a student the programme should find out their year group. The program should also display the initial of the student as well as their lastname.
Finally the program should display whether the user is a Student, a Teacher or an Admin member of staff.

TASK

Give and receive meaningful feedback - work in pairs to exchange solutions and give each other feedback on how they could be improved / optimised.



IGCSE Recap #2 - Programming paradigms

Sequence - to execute instructions one after another in a given order

Selection - to branch code depending on a condition. In Python we can use if, elif, else… or match statements.

Iteration - to execute a block of code repeatedly - either for a set number of times or until a condition is satisfied. Also called a loop.
Loops can be count-controlled, pre-condition or post-condition.

A **list** in Python is a dynamic array (we'll come back to this) which stores elements of the same data type under a single identifier.

TASK

Code “Blackjack” where the goal is to get as close to 21 as possible without going over. The computer gives you cards (random numbers). You can choose to Twist(take another card) or Stick (stop). If you go over 21, you lose.

- Keep track of how many cards the player drew.
- Let the Ace (11) also count as 1 if it would stop a bust.
- Exact 21 bonus: print a special message if the user hits 21 exactly.
- Add a scoring system across multiple rounds.
- Add a dealer: The dealer starts with a card and then keeps drawing until their total is 17 or more.

Compare the dealer’s total to the player’s to see who wins.

IGCSE Recap

Define maintainable code.

- Clear
- Easy to read
- Can be reused in other programs (by same or other programmers)

How to make code maintainable

- Comments
- Meaningful identifiers - for functions / variables etc.
- README file
- Local variables used as much as possible

Modularization

Function - A set of statements grouped together and called as needed. Functions always return at least one value.

Procedure - A set of statements grouped together and called as needed. Procedures don't return a value.

Both can be sent data in the form of parameters e.g. sum(num1, num2)

Functions and procedures can be tested separately, so programmers can use them without worrying about their functionality.

IGCSE Recap #3 - Variable scope

Local variable: exists only within the block of code where it is defined.

Global variable: exists throughout the program.

Variable scope: defines its lifetime in the program - where it can be used, where it can be modified. Depends on how and where the variables are declared.

Why is it a good idea to aim to use local instead of global variables?

Optional Tasks

Battleships

Text adventure game

Debugging and Exception handling

What are the three types of errors?

Syntax error - program will not run.

Logical error - program runs, but not as expected. requires proper testing with different test data

Runtime error - e.g. division by 0, **could cause program to crash.**

Exception handling

There are other "points of failure" for your programs.

Resources being unavailable - e.g. libraries, files.
Receiving unexpected input - e.g. a string instead of a number

These will throw what is called an "exception"

Programming languages have methods to "catch" errors and deal with them.
In Python try and except.

TASK: Edit your most recent code to use try.. Except.

Static vs dynamic data structures

A **data structure** is a way of storing and organizing data.
E.g. lists, arrays, stacks, queues …

A **static** data structure has a **predefined fixed size**. That means a **specific space in memory allocated to it** that will not change.

This means the size is set (and memory allocated) at **compile time** (before the program is run) rather than at **runtime** (whilst the program is running).

In Python, an example of a static data structure is a **tuple**.

Static vs dynamic data structures

A **dynamic data structure** doesn't require you to set the size. When it's declared, some memory is set aside to allow for elements to be added. If the list goes beyond the memory allocated, then additional memory will be allocated.

When items are removed, the memory is deallocated.

In Python, an example of a dynamic data structure is a **list**.

Static vs dynamic data structures

Flexibility

Static data structures are useful when the number of items is KNOWN (e.g. number of seats in a concert hall). If the number of items is unknown, better to use a dynamic data structure.

Memory usage:

Static data structures will use the same amount of memory even if nothing is stored in the element. Dynamic data structures are more efficient (only using the memory they need)

Speed

Static data structures are quicker to access elements directly using an index. Dynamic data structures are slower, as the memory could be fragmented (the data is stored NOT in contiguous locations)

Using lists in Python

myList = []

Useful functions are append, len, pop, sum, max, min

Use for loop to iterate through a list.

List comprehension - [expression for item in iterable]

More compact **BUT** **unreadable if too long**

Slightly faster to process

Examples: simple 1D

With conditions (even numbers)

Transformations (square numbers)

2D grids (tic-tac-toe board, chessboard, times table)

Stacks

Stacks are data structures that operate on a LIFO (Last in first out) principle. Like a stack of plates.

What might this be useful for?

Stacks support the following operations:

push(): Adds a new element on the stack.
pop(): Removes and returns the top element from the stack.
peek: Returns the top (last) element on the stack.
isEmpty: Checks if the stack is empty.
size: Finds the number of elements in the stack.

How could we implement a stack using lists?

Stacks

Stacks are data structures that operate on a LIFO (Last in first out) principle. Like a stack of plates.

What might this be useful for?

Undo / redo, back / forward buttons on browser, CPU scheduling, recursive problems, parsing, reversing a string.

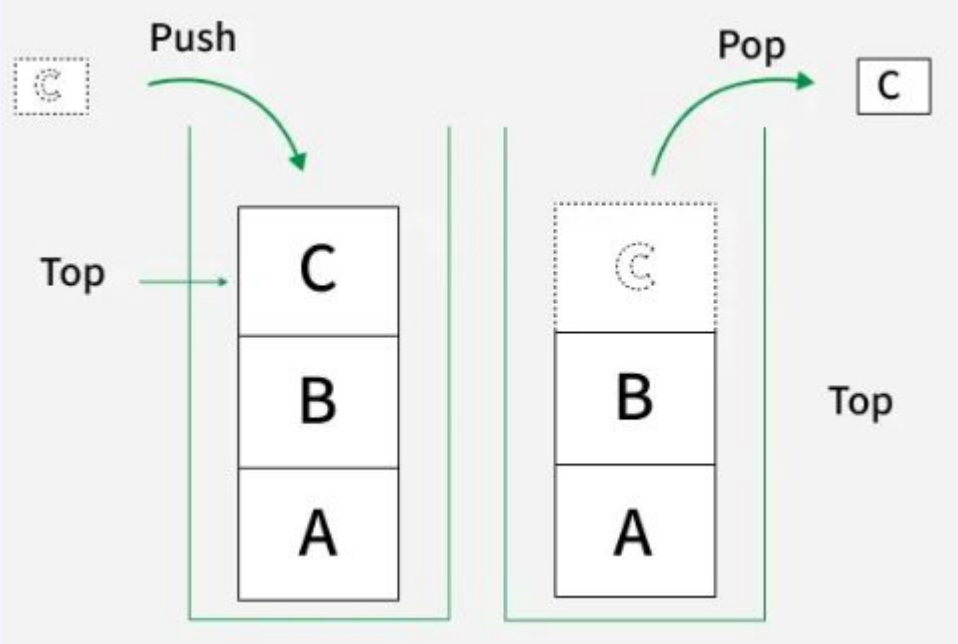
Stacks

Stacks support the following operations:

push(): Adds a new element on the stack.
pop(): Removes and returns the top element from the stack.
peek: Returns the top (last) element on the stack.
isEmpty(): Checks if the stack is empty.
size(): Finds the number of elements in the stack.

How could we implement a stack using lists?

Stacks



Stacks tasks

1. Write a function that reverses a string.

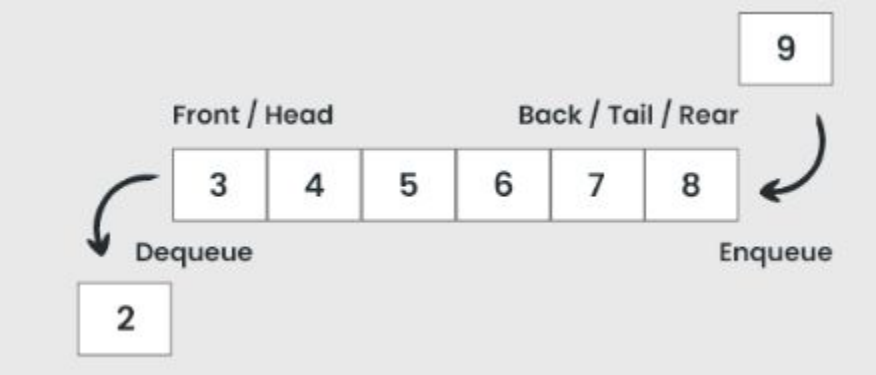
2. Write a program which takes as an input some code with different brackets, and returns True if the brackets are balanced, and False if they are not balanced.

Queues

Queues are data structures that operate on a FIFO (First in first out) principle.

Q. When could this be useful?

A. When there's a need to process tasks in order of their arrival, e.g. operating system CPU scheduling, any kind of buffer such as print queue management.



Queues

Queues support the following operations:

Enqueue: Adds an item to the queue. If queue is full, it is said to be an Overflow condition.
Dequeue: Removes an item from the queue. If the queue is empty, it is said to be an Underflow condition.
Front: Get front item from queue.
Rear: Get last item from queue.

How could we implement a queue using lists?

What's the problem?

Stacks and Queues - professional implementation

deque is a special type of list in Python that’s designed for adding/removing items quickly from both ends. Unlike a normal list, it doesn’t slow down when you take things from the front.

Under the hood, deque is built differently from lists. A list is one continuous chunk of memory, so removing from the front requires shifting everything down. A deque uses chunks (blocks) connected together, so adding/removing at either end is always fast.

Dictionaries in Python

Dictionaries are used to store data values in key:value pairs.

Written with a curly brackets. E.g.

```
lions = {"team_name": "Lions",
         "played": 0,
         "won": 0,
         "drawn": 0,
         "lost": 0,
         "points": 0
}

lions["played"] += 1
```

Main Task - shopping cart for a tuckshop

- Keep a dictionary of item names and prices.
- Track Quantities Sold
- Process Customer Orders: Take a basket of items, check if each item is valid, calculate the bill, and print a receipt.
- Handle Invalid Items: Flag items not on the menu without crashing the program.
- Update Sales: Automatically increase the sold counts when an order is placed.
- Print Daily Summary: Display units sold, revenue earned per item, and overall total earnings at the end of the day.

Extension - simulate a league table

3 main functions:

```
simulate_match(team1, team2) # calls update_table
update_table(team, result)
print_table(teams)
```

Task:

- **Make a create_team function which takes a team name and returns the dictionary (which you can then add to teams)**
- **Make all teams play each other with randomized results**
- **Print table ordered (don't do bubble sort - google how to order a dictionary based on one of the values)**

Identifying suitable data structures

ATL - thinking skills!

A list of animals in the zoo
A seating plan in a classroom
A list of planes waiting to land at an airport
A line of people waiting to pay at a shop
A representation of a shelf in a convenience store
Jobs waiting to be printed by a printer
A playlist
Your recently played songs
Hospital triage

Programming Algorithms - Big O efficiency

Programming is about developing solutions and then evaluating their **efficiency.** This includes **time complexity analysis** and **space complexity analysis.**

Both of these are expressed using **Big O notation.**

Time complexity → how the running time of an algorithm grows with input size.

Space complexity → how the memory usage grows with input size.

Time complexity

Consider the push and pop methods on a stack.

Does the number of elements in the stack affect how many operations are needed?

No, the size of the operation doesn't change with the size of the input.

This is written as O(1) - constant.

Time complexity - loops

Consider the following algorithm:

```
for i in range(5):
    print("Hello")
```

How many times will this repeat?
If we replace 5 with n, how many times will it repeat?
It will repeat n times, so the time complexity is of the order n.

This is written as O(n) and is LINEAR.

To calculate the big O notation, always take into account the worst-case scenario.

Time complexity loops

What about looping through a 2D array?

```
for i in range(5):
    for j in range(5):
        print("hello")
```

An algorithm with a double loop has quadratic complexity $O(n^2)$

An algorithm with a triple loop (three nested loops) has cubic complexity $O(n^3)$

Other big O values

Logarithmic - O(log n)

An example of this is a binary search (divide & conquer) - the search values decrease by half each iteration of the algorithm.

Log linear - O(n log n) - Quicksort (to come)

Exponential - O(c**n) - very bad. E.g. identifying possible pairings with a group of students.

Factorial - O(n!) - even worse. E.g. finding permutations of a data set.

Space complexity

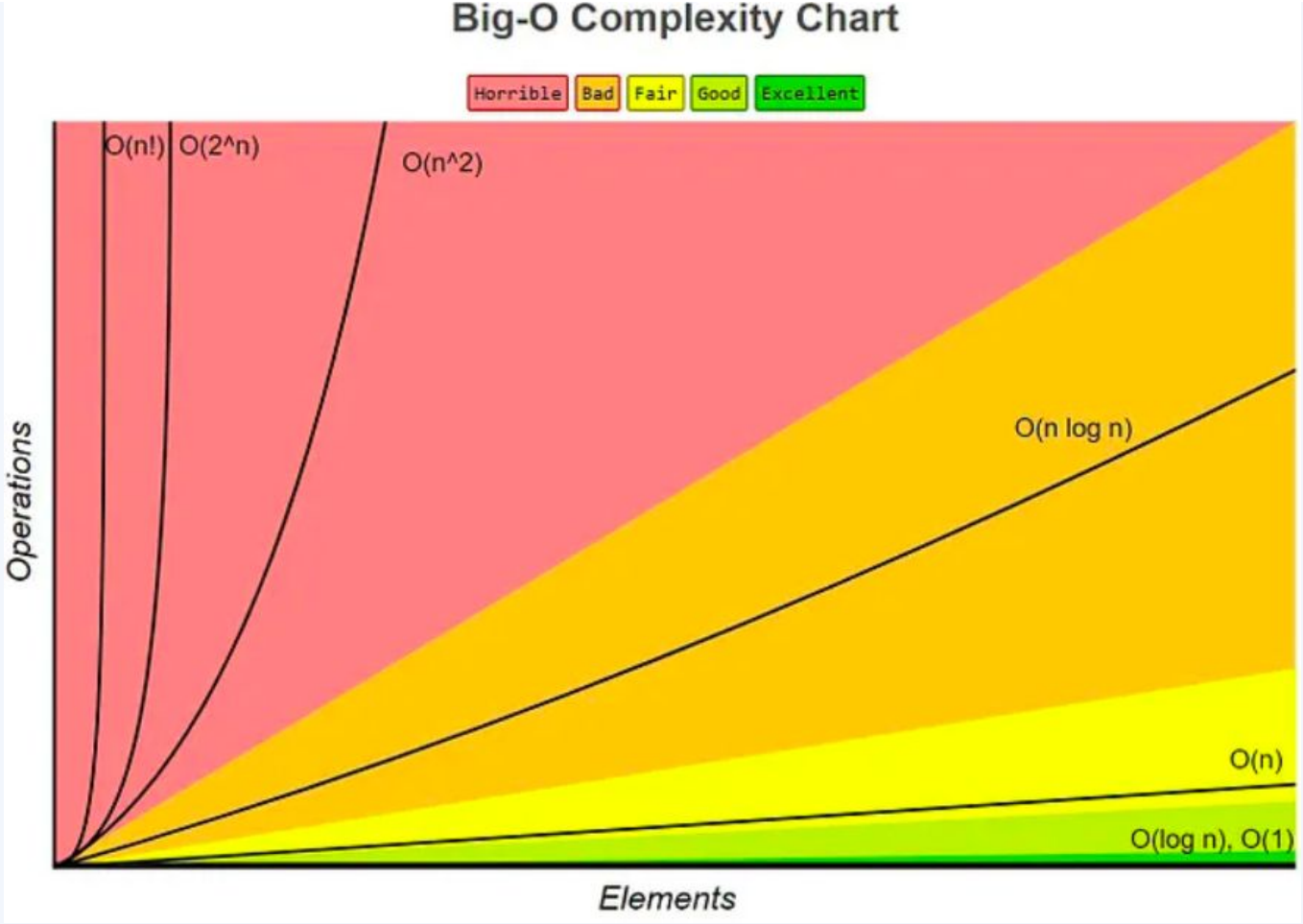
Space complexity analyses the amount of memory used by an algorithm with respect to it's input size.

Common space complexities are

O(1) - constant space
O(n) - linear space
O(n^2) - quadratic space

```
sum = 0
for i in range(n)
    sum = sum + i
print(sum)
```

For space complexity, consider variables / constants


Big-O Complexity Chart
Horrible
Bad
Fair
Good
Excellent
O(n!)
O(2^n)
O(n^2)
O(n log n)
O(n)
O(log n), O(1)
Operations
Elements

Search Algorithms

Linear search - also called "sequential search" **traverses a data structure** looking for a specific item.

Best case scenario - the item is found in the first position
Worse case scenario - the item is found in the last position

Time complexity? **O(n) (linear)**
    Because in worst case scenario we perform n checks

Space complexity? **O(1) (constant)**
    No more variables are made the more we have to search

How can the linear search be made more efficient?
Does this affect the time complexity?

Search Algorithms

Binary search - a method of searching an ordered array / list by repeatedly checking the value of the middle element and disregarding the half of the data structure that doesn't contain the searched element.

The algorithm works by

- setting variables (upper and lower) to store the upper / lower bound index.
- Calculate the middle index (mid) and compare it's value with the search key.
- Either it's the same (item found) or too low (re-assign lower variable - repeat) or too high (re-assign upper variable - repeat)
- The while condition could be while lower <= upper and not found

Search Algorithms - which is more efficient?

Time complexities are:
Linear search - O(n)
Binary search - O(log n) - faster

However, you could use **linear search** on a **small data set**, particularly where the data is not ordered. In general, simpler algorithms with higher time complexity can be acceptable for a small data set, even quadratic or cubic)

On a **larger data set, particularly where the data is already sorted**, a binary search is definitely preferable as it's more **scalable.**

Sort algorithms - bubble sort

Works by repeatedly comparing adjacent elements and swapping them if they are in the incorrect order.

Like linear search, the bubble sort algorithm can be optimised.

Code the bubble sort.
What is it's time complexity?
What is it's space complexity?

Sort algorithms - selection sort

Works by repeatedly choosing the minimum value from the unsorted portion, and placing it in the sorted portion.

The unsorted portion is initially the whole list, and the sorted portion is initially empty. After each pass, the sorted portion increases by 1 and unsorted decreases by 1.

Code the selection sort.
What is it’s time complexity?
What is it’s space complexity?

Selection Sort vs Bubble Sort

Time complexities are the same - $O(n^2)$
Space complexity is the same - O(1) constant

Bubble sort - compares adjacent elements and might swap them, so it makes MANY more swaps than insertion sort.
HOWEVER, bubble sort allows for an early exit.

Selection sort makes fewer swaps, but does not allow for an early exit.

Recursion

A technique that involves the use of functions, procedures or algorithms **calling themselves** one or more times, until specific condition/s are met.

The termination condition is also known as the **base case** - without this, the algorithm would run infinitely.

The **general case** moves towards the base case by changing its state, this is known as **winding**.

When the algorithm reaches its base case, the problem is solved / values are returned, this is known as **unwinding**.

Recursion

Recursive algorithms can provide elegant solutions when a problem is possible to be solved by repeatedly solving smaller instances of the same problem.

E.g. Factorial of a number $n! = n * (n-1)!$

E.g. Fibonacci sequence $F_n = F_{n-1} + F_{n-2}$

TASK: Use a recursive algorithm to calculate the superdigit of a number.
E.g. superdigit of 9847
9 + 8 + 4 + 7 = 28
2 + 8 = 10
1 + 0 = 1 the superdigit is 1

Recursion

When your program calls a function, the computer needs to remember where it was and what it was doing before it went into that function.

It creates a **stack frame** for this which is placed on the **call stack**

When the function returns, its frame is popped off, revealing the previous one.

In recursion, each recursive call adds another frame to the stack until the base case is reached. Then, as each call completes, the frames are popped one by one — exactly like popping plates from a stack. This is why recursion mirrors the LIFO principle.

However, if many / infinite recursive calls are made, this could potentially lead to **stack overflow**.

Recursion

Reasons to use a recursive algorithm:

- When appropriate, can be simpler and more elegant than an iterative solution.
- Allows us to break problems down into smaller more manageable tasks.

Reasons to NOT use a recursive algorithm:

- Memory overhead as a stack is required to store the recursive calls
- If you don't have a good base case you could get into an infinite loop which could create a stack overflow and crash the computer.

https://leetcode.com/problems/palindrome-number/description/

Recursion

TASK 1 - Write a function to determine if a number is a palindrome

A number is a palindrome if it reads the same forward and backward.

What is the base case? What is the general case?

TASK 2 - Given an integer array nums, return the highest common factor of the smallest number and largest number in nums. The highest common factor of two numbers is the largest positive integer that evenly divides both numbers. hcf(a,b) = hcf(b, a mod b)

Sort algorithms - Quicksort

Quicksort uses a divide and conquer approach. It makes use of a **pivot element** against which other elements in the data set are compared, to identify their correct position.

The data is partitioned into values smaller than the pivot element, and values larger than the pivot element.

This is recursively applied until all elements are sorted.

Quicksort - big O efficiency

Best case scenario is that the pivots are perfectly chosen and divide the data set into two parts.

In this case, the time complexity of the algorithm is log linear - **O(n log n)**

Because the algorithm will be applied log n times, and each time n comparisons need to be made.

Worst case scenario is that the pivots are always either the largest or smallest element. In this case it's as bad as bubble sort **$O(n^2)$**

Trace recursive algorithms - example - factorial(4)

n	Return call	Return value	Explanation
4	4 * factorial (3)	24	General case - call placed on stack - **push()**
3	3 * factorial (2)	6	General case - call placed on stack - **push()**
2	2 * factorial (1)	2	General case - call placed on stack - **push()**
1		1	Base case - now **pop()** all items off stack one by one

File processing

Storing data in a file means that the data is stored permanently - rather than in variables which are lost after your program stops running.

Text files are **sequential files**, that is the data is accessed in a linear fashion (line by line), and any **new records are added at the end**.

They are not useful for finding data quickly. But are useful for e.g.

- System log files
- Batch processing
- Storing archives

File processing

In Python you can use **with**, which automatically closes the file so you don't have to - BUT you must remember that **closing the file** is the most important process (cleaning up after yourself)

```
with open("highscore.txt", "r") as f:     # read only

with open("highscore.txt", "w") as f:     # write (will overwrite)

with open("highscore.txt", "a") as f:     # write (append)

with open("highscore.txt", "r+") as f:    # read and write
```

File processing

You can also use try and except to catch errors. Example:

```
try:
    with open("highscore.txt", "r") as f:
        highscore = f.read()
        highscore = int(highscore)
        print("The high score is",highscore)
except FileNotFoundError:
    print("Creating a new highscore.txt file")
    with open("highscore.txt", "w") as f:
        f.write("0")
    highscore = 0
```

File processing - writing

Use \n (line break) to write multiple lines to a file

```
f.write("First line\n")
```

To read from a file:

```
for line in f:
    print(line)
```

You might want to use split

```
lines = line.split(",")
```

File processing - exercises

1. Construct programming code that generates 100 random numbers with values between 1 and 70. Store those numbers into a text file, one number per line. Call the file numb.txt

2. Use the numb.txt file to read the numbers, identify the number of duplicate values and delete those duplicate values from the file. Output how many numbers were deleted.

3. Append to the same file the exact number of elements that were deleted in question 2, including values from 71 to 150.

4. Read the text file and sort the values in ascending order using an efficient bubble sort algorithm.

5. Read the values from the text file and identify the odd numbers. Store those numbers into a new text file.

File processing - exercises

1. Given a file containing daily temperature readings (one per line), calculate:
2. The average temperature.
3. The highest and lowest temperatures.
4. The number of days when the temperature increased compared to the previous day.

File processing - exercises

Exercise 2: Spell checker

Given a text document and a dictionary of valid words, determine how many words in the mystery document have been spelt correctly. Ignore casing and punctuation.

Queues

Queues are data structures that operate on a FIFO (First in first out) principle.

When could this be useful?

Any need to process tasks in order of their arrival, such as operating system CPU scheduling and any kind of buffer (area in memory to hold tasks when there is a difference in speed) such as print queue management

